SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Engineer to Excel

Accredited by NAAC A Grade

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Engineer to Excel

SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

Code of Conduct:

I Mathusri P - 192311363 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

CSA3114 - Software Testing

CO1 - AT2 - Short Answers

Mathusri P
192311363

① 3, 4, 10, 12, 20, 21, 24, 28, 29, 30

② Well defined testing process in banking system.

* a well defined testing process includes test planning, test design, execution, defect tracking and regression testing in banking System, this ensures:

* Reliability: Transaction are validated correctly
* Security: Authentication and encryption tested

Consistency: same quality across releases

Early Detection: Defects found before production

Compliance: Meets RBI / PCI - DSS Standards

Example: An ATM machine withdrawal, a defined proces tests normal withdrawal, insufficient balance, daily limit exceeded network failure and concurrent transactions - preventing fields. failures that would otherwise reach customer

④ Error, Defects, failure classification

* Error: Human mistake. Eg: Developer writes = instead of ==

Defect: Flaw in code Eg: wrong operator in code

Failure: Showing output during execution Eg: Any password accepted.

Relationship: Error → Defect → Failure.

(9) Example. mismatch in output = Failure;
cause in Code = Defect
human mistake = Error.

(10) Functional defect in ATM transaction.

* This is a functional defect - system does not perform its intended function

* Causes: Incorrect transaction login (wrong balance deduction, wrong dispense amount).

Prevention:

→ clear requirement analysis
→ Decision table and state transition testing
→ Test Cases: valid withdrawal, insufficient balance, daily limit, network failure.
→ Integration and regression testing

Example:

ATM dispenses cash but does not debit account.
Functional Defect.

Performance Defect classification

* This is a performance defect (non-functional), specifically load/stress defect.

* (Classification Techniques) By type, severity, priority, phase, root cause.

* (Why performance Defect) System works functionally but fails under heavy load (slow response, timeout).

* (How classification helps)
  - → align to right team (Performance engineers)
  - → uses right tools (JMeter, LoadRunner)
  - → Guide root causes (DB queries, memory leaks)
  - → Prioritizes based on user impact

* (Example:) E-commerce site slow at 10,000 user-load test and caching fix it.

(20) Testing as Engineering activity

* Testing must be planned, systematic and documented - not random

*(Phases)

① Requirement analysis
② Test planning
③ Test Design (EP, BVA, decision Table)
④ Test execution
⑤ Defect Tracking
⑥ Metrics and review

*(Importance of Systematic Design:)

→ full coverage
→ Repeatable and traceable
→ Find more defects
→ Supports automation
→ Measurable quality

*(Example:) Login module tested to valid invalid, boundary, and security inputs - unlike random testing

(21) Data inconsistency in healthcare integration

*(Root causes:)

→ Interface mismatch
→ Timining / concurrency issues
→ Transaction failures

validation gaps

→ Incorrect field mapping

* (Testing principles applied:) Integration testing, interface testing, end to end testing, database testing, concurrency testing

* (Example:) Patient admission updates patient table but not Bed allocation table - integration testing catches it

* (How proper) (testing ensures consistency)

→ Validates data flows between modules

→ Verifies ACID properties

→ Checks data formats and contracts

→ Ensures grateful error handling

(24) Defect Repository for tracking & analyses

* (Defect repository) Centralized database of all defects (ID, severity, priority, status, root cause, fix)

* (Tracking benefits)

→ unique ID for each defect

→ status tracking (New → fixed → closed)

→ severity / Priority for fixing order

→ Traceability to requirments

Analysis benefits:

→ Trend and module analysis
→ Root cause analysis
→ Identifies defect prone modules
→ Improves test case design
→ Support regression testing

Example: 60% defects in payment module → focus testing there.

(25) structured testing for banking reliability.

→ structured testing = planned + systematic + documented steps.

① Requirement analysis - clear transaction rules
② Test Planning - scope, risk, resource
③ Test Design - EP, BVA, decision table.
④ Testing levels - unit, integration, system, UAT
⑤ Non-functional - performance security
⑥ Regression Testing
⑦ Defect Tracking

Example: concurrent fund transfer tested - prevent failure.

Enhance reliablity
→ full coverage
→ Early defect detection
→ Consistent quality
→ security verified
→ meets regulations

(29) Defect leakage root causes & prevention

(Root causes) poor coverage, weak test design, late testing, no regression, poor defect tracking, no review, environment mismatch.

Prevention: shift left testing comprehensive test design regression testing defect respository, reviews production like environment

Example: Missing integration test in payment module

(20) Early defect detection & efficiency

Principles: Early testing, shift left, prevention over detection, review

Strategy: Requirment (reviews) → Design (tests, Development
Integration
System performance

Improvement Efficiency: cost reduction
less rework
fewer production defect
better customer satisfaction.

Example: Requirment Defect in review cost $100, in production cost $10,000

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand